

md2D / md3D Optimization Report

What made the codes faster, how each change was checked, and one numerical correction found along the way. Every step kept the full test suite passing and was measured with the same benchmark.

Branch `optimization` (merged into `master`) · 8-core x86-64 PC, Debian · measurements in `tests/bench_history.jsonl`

3–20×

faster on the benchmark runs

128

automated tests, all passing

$\leq 10^{-9}$

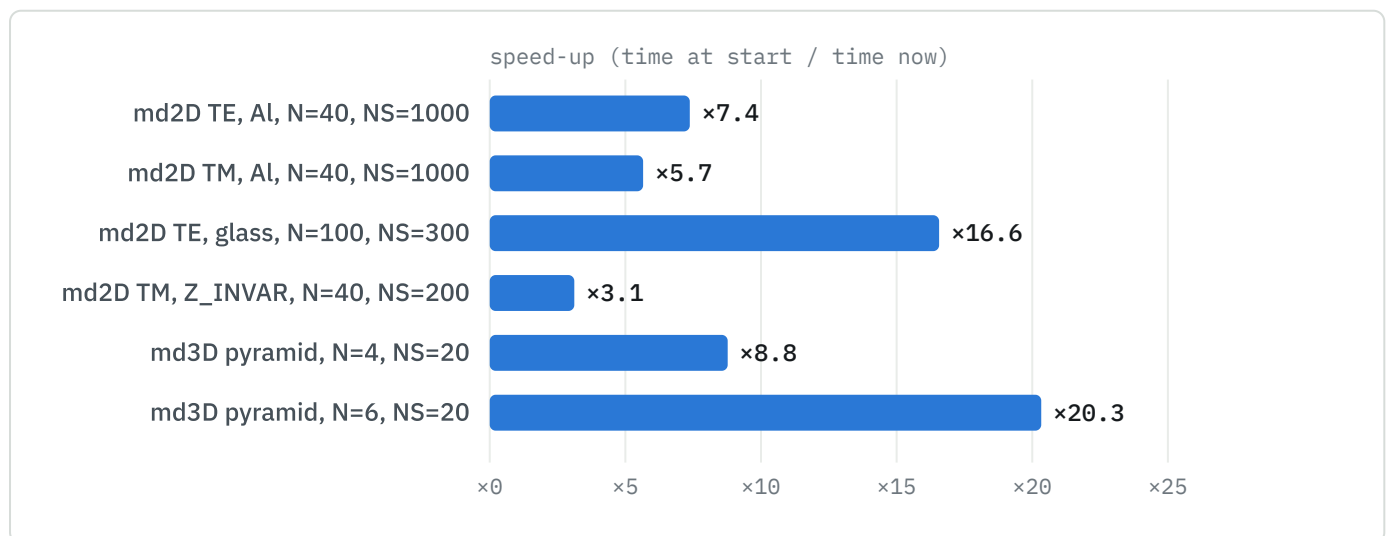
change of results from the speed-ups

exact

flat interfaces with RK4 after the slice-boundary fix

1 Summary

Six configurations are timed by `make bench`. From the starting point to now, they run 3–20 times faster; the largest gains are on the biggest problems. Most of the gain comes from two changes: using OpenBLAS instead of the reference BLAS, and no longer reallocating large work matrices at every slice. The speed-ups did not change any result beyond rounding.



The `Z_INVAR` benchmark gains least (x3.1). It computes each slice through an eigen-decomposition of M rather than RK4, a path none of the changes targeted and that has not been profiled yet.

2 How the work was checked

- **Tests before speed.** An automated test suite was built first (`make test` , about 110 checks in under a minute; `make test-validation` , the thesis tables at reduced resolution in about 1.5 min). It compares with stored reference results to a relative 10^{-9} , with exact solutions (Fresnel, thin-film Airy formula) to 10^{-12} , checks physical laws (energy conservation, reciprocity, symmetries) and that md2D and md3D agree where both apply. Each optimization had to pass all of it.
- **Benchmark.** `make bench` records wall and CPU time of the six configurations, the BLAS library, the thread count and the machine load before the run, with the commit, in `tests/bench_history.jsonl`. The machine was shared (load 1.7–3.2), so single-threaded steps are compared on CPU time, which barely depends on the load, and differences below ~5 % are treated as noise.

- **Profile.** valgrind's callgrind showed where the time goes before any code was touched (§ 3).

3 Where the time went

One routine dominated: `zgemv`, the complex matrix-matrix product of BLAS, took **86–96 %** of the run time in md2D TE, md2D TM and md3D. The programs' own code was a few percent. By caller:

SHARE OF RUN TIME	MD2D TE	MD2D TM	MD3D
RK4 step of a slice (<code>rk4_P_matrix</code>)	82 %	91 %	59 %
... of which building M (Fourier, Toeplitz, products)	1.5 %	53 %	14 %
rest of the T-matrix computation	~1 %	~1 %	~30 %
S-matrix recursion and inversions	~17 %	~8 %	~9 %

Two conclusions followed: a faster BLAS would help everything at once, and within the code the RK4 step was the place to look.

4 The steps

Times in seconds for each benchmark after each step; the small figure is the change from the previous row. Rows up to "RK4 work matrices" are single-threaded CPU times; the last two rows are wall times with automatic threads.

STEP	COMMIT	MD2D TE AL, N=40, NS=1000	MD2D TM AL, N=40, NS=1000	MD2D TE GLASS, N=100, NS=300	MD2D TM Z_INVAR, N=40, NS=200	MD3D PYRAMID N=4, NS=20	MD3D PYRAMID N=6, NS=20
Starting point: reference BLAS, md3D built with -g	0453a8b	32.6	63.7	146.1	27.2	9.30	92.9
md3D built with -O3	f26e652	32.6 +0%	63.7 +0%	146.1 +0%	27.2 +0%	9.30 +0%	87.6 -6%
OpenBLAS instead of the reference BLAS (1 thread)	system	6.78 -79%	14.3 -77%	24.9 -83%	8.58 -68%	1.67 -82%	10.4 -88%
Matrix product without copies	161cb71	6.06 -11%	13.2 -8%	23.8 -4%	9.22 +7%	1.56 -7%	10.2 -2%
RK4 work matrices kept between slices	501ac73	4.09 -33%	10.8 -18%	18.4 -23%	8.76 -5%	1.50 -4%	9.66 -5%
Automatic OpenBLAS thread count (wall time)	68b69d8	4.65 +14%	10.5 -3%	8.90 -52%	8.73 -0%	0.90 -40%	4.37 -55%
One-sided sampling at slice boundaries (wall time)	d76dc21	4.42 -5%	11.3 +7%	8.82 -1%	8.73 +0%	1.06 +18%	4.57 +5%

md3D built with -O3 f26e652

md3D was compiled with debugging flags only (its `-O3` line was commented out). Switching was only safe after a memory bug was fixed: for index-map structures md3D read an uninitialised pointer, which the debug build survived by chance and an optimized build turned into a crash (fixed in `5903b05`, results unchanged). Gain: 6 % at N = 6, since most of the time is in BLAS.

OpenBLAS

Replacing the reference BLAS by OpenBLAS needs no rebuild: on Debian, installing `libopenblas0-pthread` switches `libblas.so.3` and `liblapack.so.3` system-wide. All tests pass with it (results within 10⁻⁹). Single-threaded, the benchmarks run **3.2–8.4 times faster**.

Matrix product without copies **161cb71**

Each product allocated two temporary matrices, copied and transposed both inputs, and transposed the result, to bridge C's row-major storage and Fortran's column-major. A row-major matrix is its own transpose in column-major order, so $C = A \cdot B$ is obtained directly as $C^T = B^T \cdot A^T$ by passing B and A in swapped order. The old path is kept, and taken automatically, if the output overlaps an input or a matrix is not stored contiguously. Gain: 2–11 %.

RK4 work matrices kept between slices **501ac73**

The RK4 step allocated and freed seven $2V \times 2V$ complex matrices per slice in md2D (three in md3D). Each is a few hundred kilobytes, so every allocation went to the operating system and every page was zeroed again. The matrices are now allocated once per run. Results are bit-identical; md2D gains **18–33 %**.

Automatic OpenBLAS thread count **68b69d8**

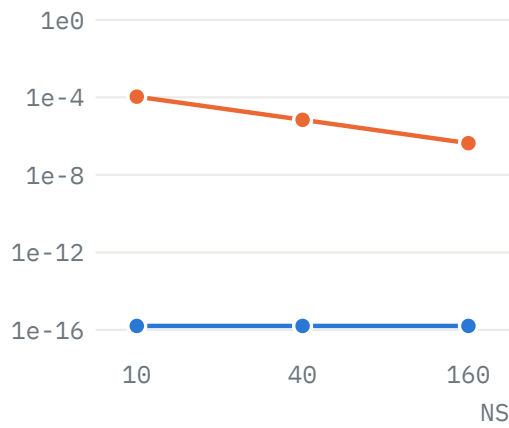
OpenBLAS uses every core by default, which made md2D's small matrices slower. Measurements showed one thread is best below a matrix dimension of about 300, and four threads are 1.5–2.3 times faster above ~400. The programs now choose (1 or 4, never more than the cores) unless `OPENBLAS_NUM_THREADS` or `OMP_NUM_THREADS` is set; the OpenBLAS functions are weak references, so the programs still link and run with any BLAS. Wall time: md2D N = 100 from 18.5 to 8.9 s, md3D N = 6 from 9.7 to 4.4 s.

5 An accuracy correction: sampling at slice boundaries **d76dc21**

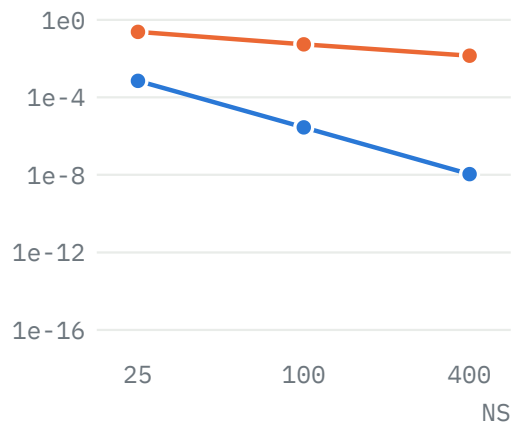
While testing the idea of reusing M between consecutive slices, three reference results changed: in those cases a profile sample or an interface lies exactly on a slice boundary, where the permittivity is undetermined. The code chose the material by the floating-point rounding of $z \pm \Delta z/2$.

Each S-matrix slice is integrated on its own, between two zero-thickness layers where the field is re-expressed in plane waves. The integration step of a slice must therefore see the structure as the limit from inside that slice: from above at its bottom, from below at its top. Both programs now do so (a `z_side` flag read by the structure functions). The effect on structures whose interfaces lie on slice boundaries:

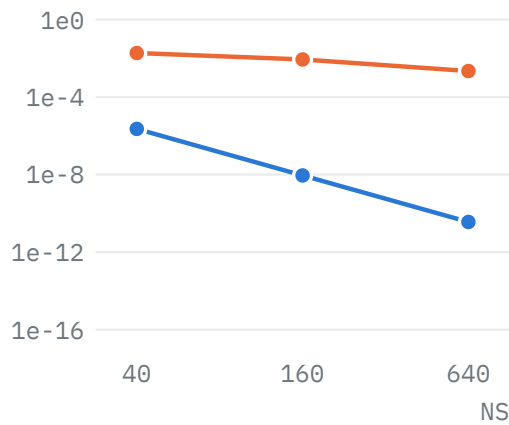
Flat interface vs Fresnel



Lamellar layer (TE) vs exact



Layered index map vs exact



— before (rounding decides) — one-sided rule

- A flat interface is now exact at any number of slices (the 10^{-16} points are rounding), in md2D and md3D.
- Lamellar layers and layered index maps converge at fourth order in NS instead of first.
- Smooth profiles are unaffected in accuracy: both versions converge to the same limit (7×10^{-7} apart at NS = 80 000).
- Agreement with the thesis is unchanged for smooth structures and better for the lamellar grating in conical incidence (case 06: $1.3 \times 10^{-2} \rightarrow 3.2 \times 10^{-3}$ against GSolver).
- The stored reference results of RK4 cases were regenerated; they moved within their discretization error. Speed is unchanged.

6 Other findings

- RK4 converges only at about second order in NS on smooth profiles: the profile crosses the slices, so the permittivity is discontinuous inside them.
- `Z_INVAR` with a single thick slice overflows at large N (evanescent orders grow like $e^{2\pi N t/L}$ inside a slice of thickness t); the manual now gives the rule $NS \geq N \cdot h/L$.
- In TM the energy balance has a small floor (about 3×10^{-6} in the test case) set by the finite-N factorization, so it measures NS convergence only in TE.
- `fft_filter` acts only in TM and `smoothing` only on stacks; both are now documented.

7 What could come next

IDEA	EXPECTED EFFECT	NOTES
Reuse M at a slice boundary when both sides sample the same material	~20 % in md2D TM, less elsewhere	now well defined thanks to the one-sided rule; cheap check of the sampled permittivity; results bit-identical
Compute the slices in parallel	up to the number of cores for md2D	the T matrices of the slices are independent; only the S-matrix recursion is sequential (~10–17 %)
Exploit the block structure of M in TE	several × on the TE RK4 step	one block of M is a multiple of the identity in md2D TE
Profile md3D's T-matrix products outside RK4	up to ~30 % of md3D	not yet analysed
Faster <code>Z_INVAR</code>	unknown	least improved so far (×3.1); not yet profiled

8 Reproducing the measurements

```
make                # build
make test           # fast tests (~40 s with OpenBLAS)
make test-validation # thesis tables at reduced resolution (~1.5 min)
make bench          # timings, appended to tests/bench_history.jsonl
OPENBLAS_NUM_THREADS=1 make bench # single-thread CPU comparison
```

Benchmarks from a loaded machine are comparable on CPU time only; the load before each run is stored with it.